Advanced Topics

Jungtaek Kim (jtkim@postech.ac.kr)

Machine Learning Group,
Department of Computer Science and Engineering, POSTECH,
77 Cheongam-ro, Nam-gu, Pohang 37673,
Gyeongsangbuk-do, Republic of Korea

February 28, 2019

Table of Contents

Prototypical Networks for Few-Shot Learning

Anomaly Detection Using Generative Adversarial Networks

Automated Machine Learning
- Automated Machine Learning
- Issues on Automated Machine Learning
- Previous Works

Prototypical Networks for Few-Shot Learning

Few-Shot Classification

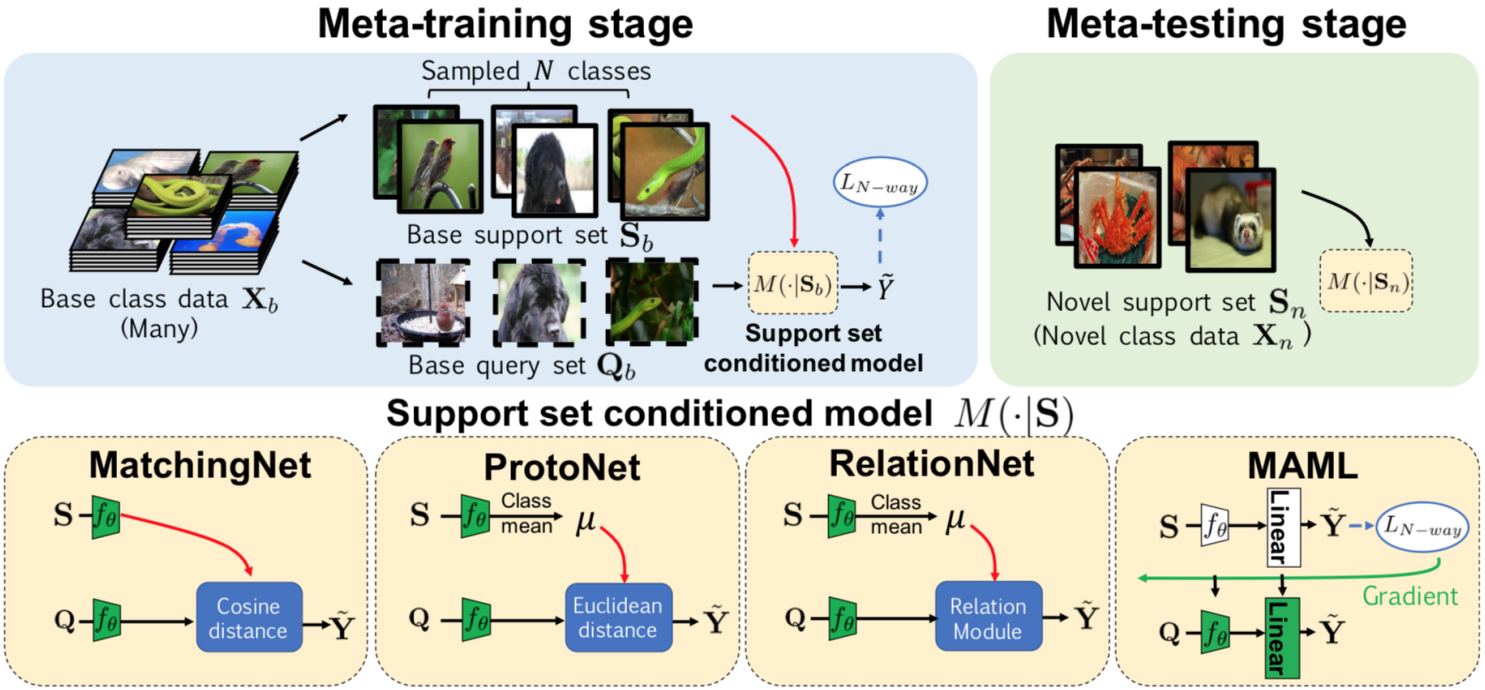


Figure 2: **Meta-learning few-shot classification algorithms.** The meta-learning classifier $M(\cdot|\mathbf{S})$ is conditioned on the support set $\mathbf{S}$. (*Top*) In the meta-train stage, the support set $\mathbf{S}_b$ and the query set $\mathbf{Q}_b$ are first sampled from random N classes, and then train the parameters in $M(.|\mathbf{S}_b)$ to minimize the N-way prediction loss $L_{\text{N-way}}$. In the meta-testing stage, the adapted classifier $M(.|\mathbf{S}_n)$ can predict novel classes with the support set in the novel classes $\mathbf{S}_n$. (*Bottom*) The design of $M(\cdot|\mathbf{S})$ in different meta-learning algorithms.

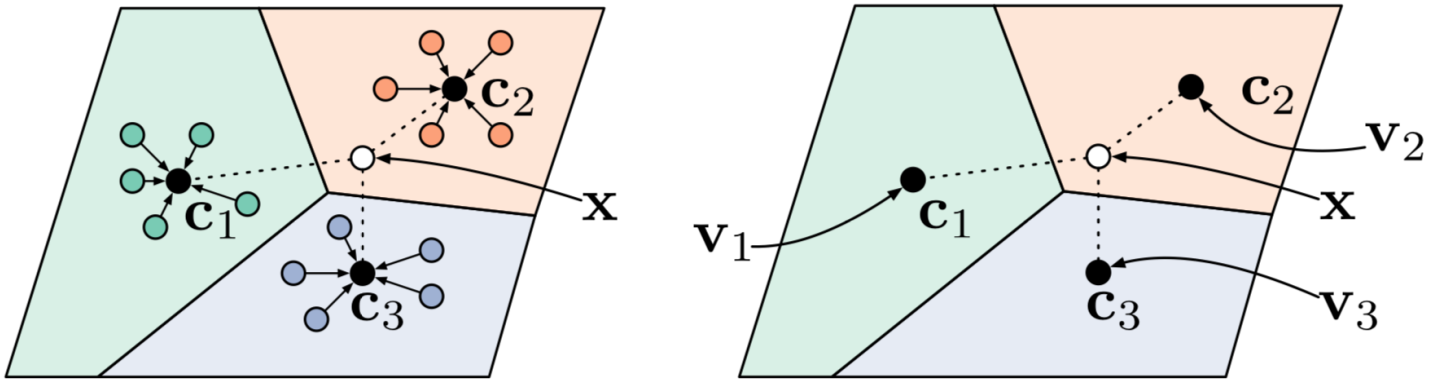
Figure 1: `https://openreview.net/forum?id=HkxLXnAcFQ`

Prototypical Networks for Few-Shot Learning

- For the network trained using K shots of N classes, prototypical network classifies queries into one of N classes.
- It learns a metric space by computing distances to prototype representations of each class.
- Prototype vectors are determined by the arithmetic mean of embedding vectors of support set.

- `https://arxiv.org/abs/1703.05175`

Prototypical Networks for Few-Shot Learning



(a) Few-shot (b) Zero-shot

Figure 1: Prototypical networks in the few-shot and zero-shot scenarios. **Left**: Few-shot prototypes $\mathbf{c}_k$ are computed as the mean of embedded support examples for each class. **Right**: Zero-shot prototypes $\mathbf{c}_k$ are produced by embedding class meta-data $\mathbf{v}_k$. In either case, embedded query points are classified via a softmax over distances to class prototypes: $p_{\boldsymbol{\phi}}(y = k|\mathbf{x}) \propto \exp(-d(f_{\boldsymbol{\phi}}(\mathbf{x}), \mathbf{c}_k))$.

Prototypical Networks for Few-Shot Learning

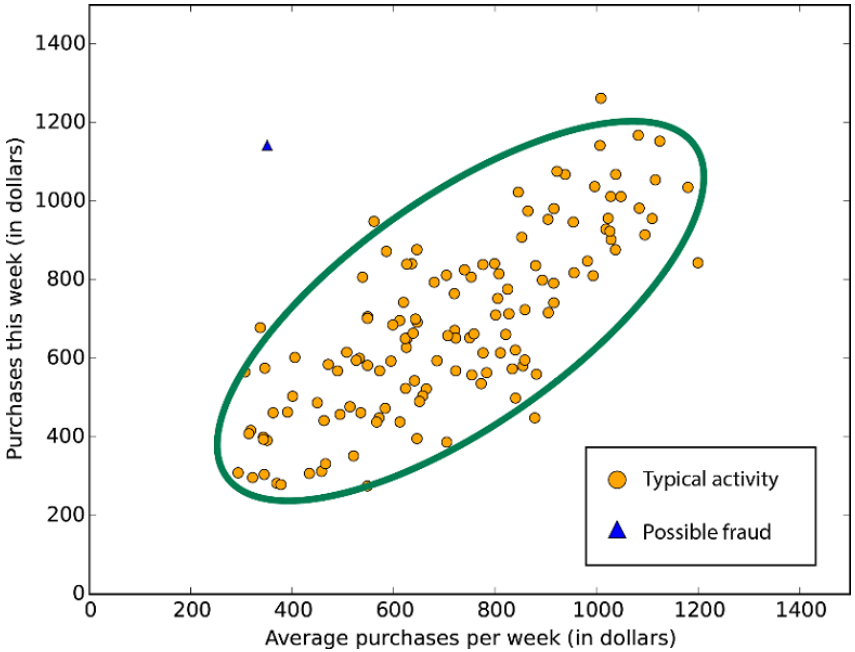
Table 1: Few-shot classification accuracies on Omniglot.

Model	Dist.	Fine Tune	5-way Acc. 1-shot	5-way Acc. 5-shot	20-way Acc. 1-shot	20-way Acc. 5-shot
MATCHING NETWORKS [29]	Cosine	N	98.1%	98.9%	93.8%	98.5%
MATCHING NETWORKS [29]	Cosine	Y	97.9%	98.7%	93.5%	98.7%
NEURAL STATISTICIAN [6]	-	N	98.1%	99.5%	93.2%	98.1%
PROTOTYPICAL NETWORKS (OURS)	Euclid.	N	**98.8%**	**99.7%**	**96.0%**	**98.9%**

Anomaly Detection Using Generative Adversarial Networks

Anomaly Detection

- It tries to identify anomalies without labels.
- Usually, it is solved by thresholding and density estimation



AnoGAN

- AnoGAN solves anomaly detection in the unsupervised manner.
- It finds anomalies using a manifold of normal anatomical variability.
- DCGAN is used to train.

- `https://arxiv.org/abs/1703.05921`

AnoGAN

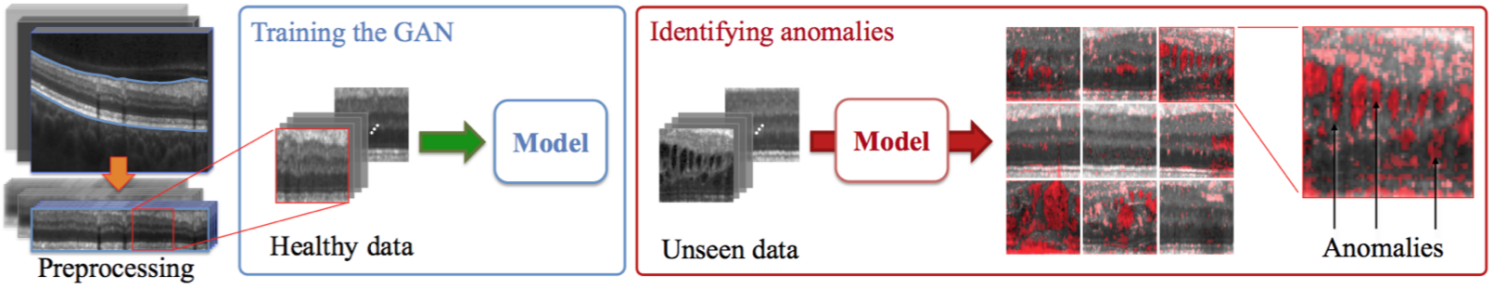


Fig. 1. Anomaly detection framework. The preprocessing step includes extraction and flattening of the retinal area, patch extraction and intensity normalization. Generative adversarial training is performed on healthy data and testing is performed on both, unseen healthy cases and anomalous data.

AnoGAN

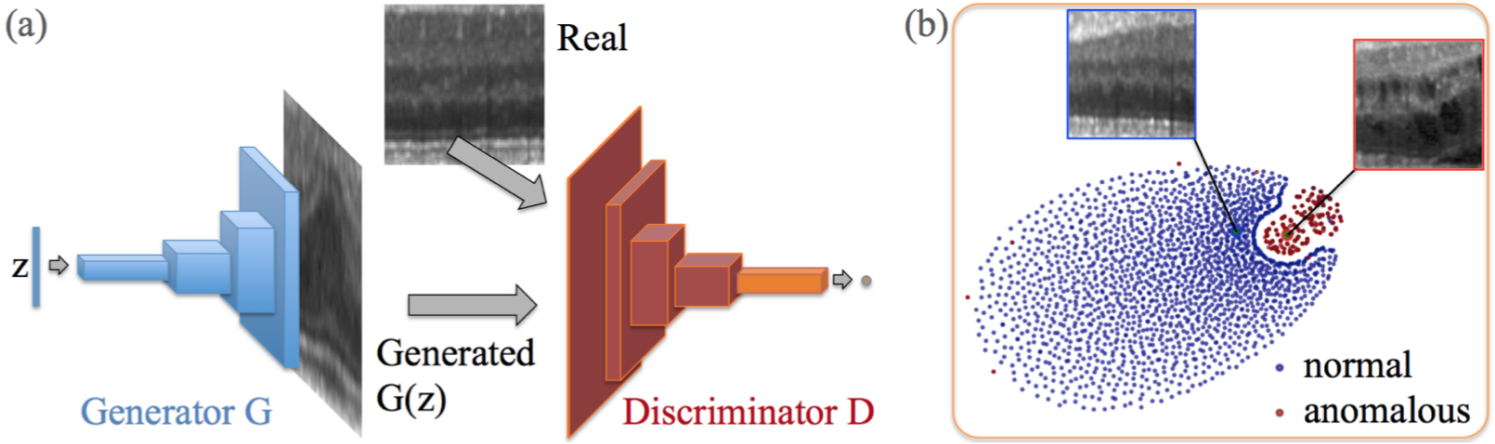


Fig. 2. (a) Deep convolutional generative adversarial network. (b) t-SNE embedding of normal (blue) and anomalous (red) images on the feature representation of the last convolution layer (orange in (a)) of the discriminator.

AnoGAN

- The loss function for additional training steps is

$$\mathcal{L}(\mathbf{z}) = (1-\lambda)\mathcal{L}_R(\mathbf{z}) + \lambda\mathcal{L}_D(\mathbf{z}) \tag{1}$$

  where $\mathcal{L}_R(\cdot)$ is a residual loss and $\mathcal{L}_D(\cdot)$ is a discrimination loss.

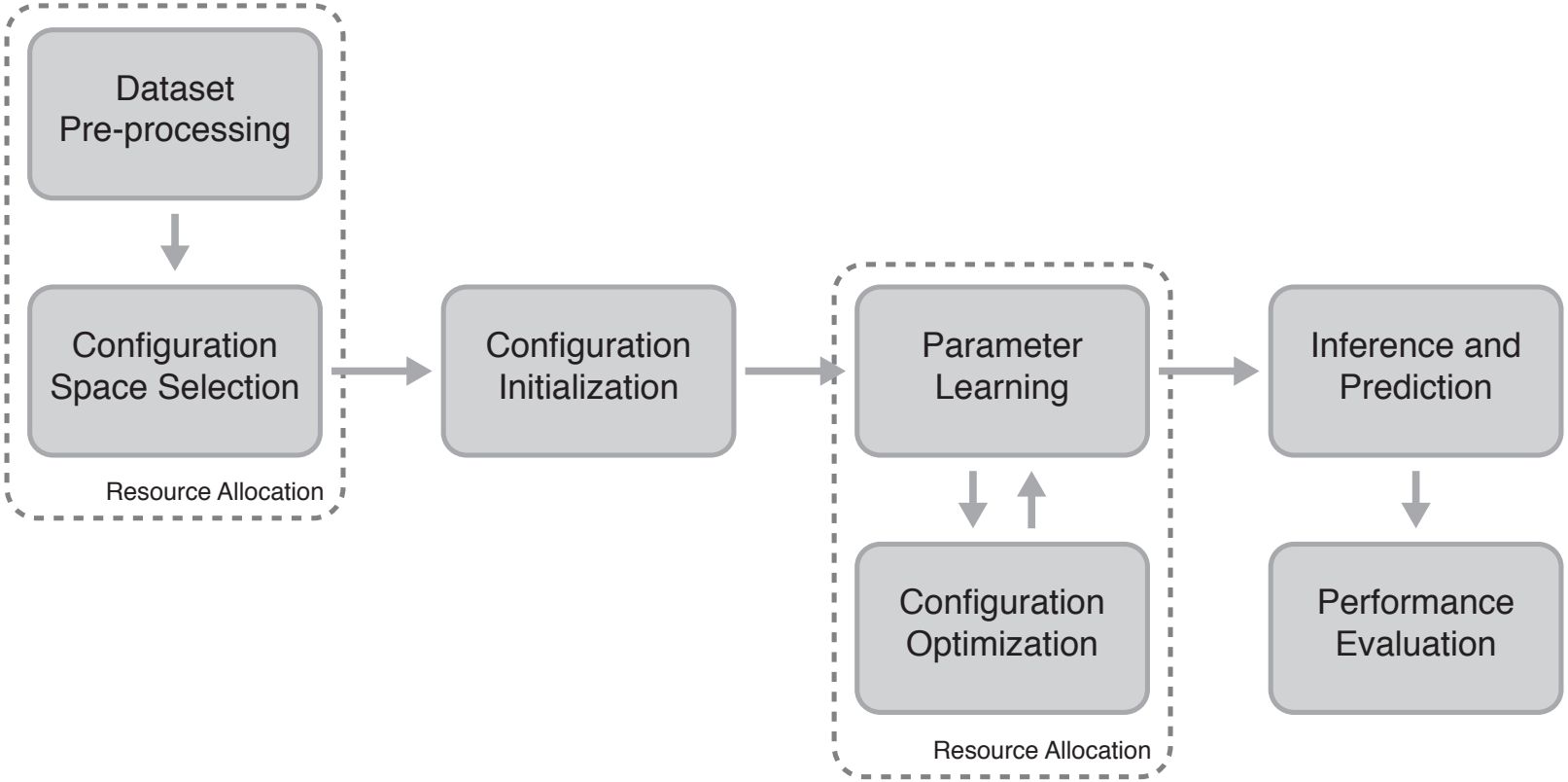
- Similarly, an anomaly score is defined as

$$A(\mathbf{x}) = (1-\lambda)R(\mathbf{x}) + \lambda D(\mathbf{x}) \tag{2}$$

  where each score corresponds each loss at the last update iteration.

Automated Machine Learning

Automated Machine Learning



Automated Machine Learning

- Data pre-processing
- Configuration space selection
  - Specify a searching space $\mathcal{S}$ (e.g., a Cartesian space of kernel type, penalty parameter, kernel coefficient, and degree)
- Configuration initialization
  - Random initialization
  - Low discrepancy initialization
  - Learn to initialize
- Configuration optimization
  - Random search
  - Grid search
  - Bayesian optimization
  - Neural architecture search
- Resource allocation

Automated Machine Learning

- Attempt to find automatically the optimal machine learning model without human intervention.
- Usually include feature transformation, algorithm selection, and hyperparameter optimization.
- Given a training dataset $\mathcal{D}_{\text{train}}$ and a validation dataset $\mathcal{D}_{\text{val}}$, the optimal hyperparameter vector $\boldsymbol{\lambda}^*$ for an automated machine learning system:

$$\boldsymbol{\lambda}^* = \mathrm{AutoML}(\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \Lambda)$$

where AutoML is an automated machine learning system and $\lambda \in \Lambda$.

Issues on Automated Machine Learning

- High-dimensional searching space
- Categorical variables
- Uncertainty estimation
- Performance evaluation without re-training
- Efficient automated machine learning
- Automated machine learning for neural networks
- Parallelism for automated machine learning
- Initialization for automated machine learning
- Resource allocation for automated machine learning

Previous Works

- Bayesian optimization and hyperparameter optimization
  - `GPyOpt` [The GPyOpt authors, 2016]
  - `SMAC` [Hutter et al., 2011]
  - `BayesOpt` [Martinez-Cantin, 2014]
  - `bayeso`
  - SigOpt API [Martinez-Cantin et al., 2018]
- Automated machine learning framework
  - `auto-sklearn` [Feurer et al., 2015]
  - `Auto-WEKA` [Thornton et al., 2013]
  - Our previous work [Kim et al., 2016]

Thank you.

References I

M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems (NIPS)*, pages 2962–2970, Montreal, Quebec, Canada, 2015.

F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *Proceedings of the International Conference on Learning and Intelligent Optimization*, pages 507–523, Rome, Italy, 2011.

J. Kim, J. Jeong, and S. Choi. AutoML Challenge: AutoML framework using random space partitioning optimizer. In *International Conference on Machine Learning Workshop on Automatic Machine Learning (AutoML)*, New York, New York, USA, 2016.

R. Martinez-Cantin. BayesOpt: A Bayesian optimization library for nonlinear optimization, experimental design and bandits. *Journal of Machine Learning Research*, 15:3735–3739, 2014.

R. Martinez-Cantin, K. Tee, and M. McCourt. Practical Bayesian optimization in the presence of outliers. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, Lanzarote, Spain, 2018.

The GPyOpt authors. GPyOpt: A Bayesian optimization framework in Python, 2016. `https://github.com/SheffieldML/GPyOpt`.

C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 847–855, Chicago, Illinois, USA, 2013.